

Airbnb NYC Open Data Data Cleaning in MySQL

Turning 26 Columns of Messy Raw Data Into an Analysis-Ready Dataset

Author: Dev | M.Sc. Statistics, University of Delhi

Dataset: Airbnb Open Data — New York City (Kaggle) | 26 Columns Processed

Tool: MySQL | Staging Tables: 6 | Techniques: Window Functions, LAG(), COALESCE, STR_TO_DATE

Output: Fully cleaned, analysis-ready table with no duplicates, consistent types, no invalid values

A non-technical summary of data quality decisions and business value for analytics and product stakeholders.

1. What Problem Were We Solving?

Real-world datasets are rarely clean. When Airbnb listing data is exported from a platform and stored in a CSV file, it arrives with a mixture of formatting inconsistencies, impossible values, duplicate records, missing information, and currency symbols embedded in numeric columns. Before any meaningful analysis can happen — pricing trends, neighbourhood comparisons, host behaviour patterns — these issues must be resolved systematically.

This project took the raw Airbnb New York City Open Data dataset and cleaned every one of its 26 columns using MySQL, a professional database management tool. The result is a fully standardised, analysis-ready table that any data analyst, business intelligence tool, or machine learning pipeline can use immediately, with confidence that the data is trustworthy.

What	Detail
Dataset	Airbnb Open Data — New York City listings (Kaggle)
Raw columns	26 covering listing details, pricing, host info, location, and reviews
Tool used	MySQL (professional relational database)
Approach	Six sequential staging tables — each fixing one layer of problems
Final output	23 clean columns, correct data types, no duplicates, no impossible values
Key techniques	Window functions, forward-fill imputation, context-aware logic, date parsing, currency parsing

2. The Dataset — What Information It Contains

The dataset describes individual Airbnb listings in New York City. Each row is one listing, and the 26 columns cover six categories of information:

Category	Columns Included
Listing identity	Listing ID, listing name, room type
Host information	Host ID, host name, host identity verification status
Location	Neighbourhood group (borough), neighbourhood, latitude, longitude, country, country code
Booking & policy	Instant bookable flag, cancellation policy, minimum nights, house rules, license
Pricing	Price per night, service fee
Reviews & activity	Number of reviews, last review date, reviews per month, review rating, host listings count, availability (days/year)

Three columns — host name, house rules, and license — were removed entirely during cleaning. Host names are personal identifiers with no analytical value; house rules are unstructured free text that cannot be meaningfully analysed without natural language processing; and the license column was largely empty or non-standard. The final clean table retains 23 columns.

3. How We Approached It — The Six-Stage Pipeline

Rather than modifying the raw data directly — which would make mistakes irreversible — we built a chain of six separate tables, each inheriting the previous one's output and fixing the next layer of problems. This is considered best practice in professional data engineering: the original raw file is never touched, and any stage can be inspected or restarted without re-importing the data.

#	Stage Name	What Was Fixed
Raw	Original import	The CSV file loaded as-is. No modifications made. Preserved permanently as the rollback point.
1	Deduplication	Identified and removed all exact duplicate rows using a window function. The listing ID was confirmed unique and promoted to primary key.
2	Host identity fixes	Standardised the host verification status column (two inconsistent text formats merged into one). Host name column removed.
3	Neighbourhood fixes	Missing borough names filled from neighbouring rows. Listings with no neighbourhood at all deleted. Coordinates rounded for consistency. Country and country code defaulted to US values.
4	Booking policy fixes	Missing instant-bookable flags filled from neighbouring rows. Missing cancellation policies inferred from booking type.
5	Numeric, date & final	Currency symbols stripped from price columns. Dates parsed from text to proper date format. Future-dated reviews deleted. Remaining blank strings converted to NULL. All helper columns dropped.

The raw source table was never modified. Every change was made in a new table. This means any cleaning decision can be reviewed, reversed, or rebuilt from scratch at any point — a professional-grade safeguard that prevents data loss.

4. The Problems Found — And What We Did About Them

Here is a plain-English account of every major data quality issue found across the 26 columns and the specific decision made to resolve it:

Problem Found	Column(s) Affected	Decision & Reasoning
---------------	--------------------	----------------------

Exact duplicate rows	All 26 columns	Removed — only the first occurrence of any repeated record was kept. Duplicates add no information and inflate analysis counts.
Inconsistent text formats ('verified' vs 'Verified', '1' vs 'TRUE')	Host verification, instant bookable	Standardised to consistent labels. Mixed formats make grouping and filtering unreliable.
Missing borough/neighbourhood values	Neighbourhood group, neighbourhood	Borough: filled from the previous row (neighbouring listings often share the same borough). Neighbourhood: rows deleted if missing — cannot infer the specific neighbourhood reliably.
Missing host verification status	host_identity_verified	Filled from the previous row. Adjacent listings frequently share host characteristics.
Currency symbols in numeric columns (e.g. '\$1,200')	Price, service fee	Dollar signs and comma separators stripped; columns converted to integer type. Numeric columns must be numbers, not formatted strings.
Dates stored as text strings (e.g. '3/15/2023')	Last review date	Converted to proper DATE format. Text dates cannot be sorted, compared, or used in date calculations.
Future-dated review records	Last review date	Deleted. A review date in the future is logically impossible and indicates a data entry error.
Empty strings treated as values	Reviews per month, review rate, availability, listings count	Converted to NULL. An empty string is not the same as zero — it means the information is missing. Leaving it as empty string corrupts averages and counts.
Prices stored as text with no currency symbol	Price, service fee (residual blanks)	Set to NULL after stripping. Missing prices are genuinely unknown and should not default to zero.
Coordinates with excessive decimal places	Latitude, longitude	Rounded to 2 decimal places. Excessive precision is unnecessary for neighbourhood-level geographic analysis.
Missing country and country code	Country, country code	Defaulted to 'United States' and 'US'. All listings are US-based NYC properties — this is a safe factual default.
Missing cancellation policy	Cancellation policy	Inferred from booking type: instantly bookable listings → flexible policy; non-instantly bookable → strict policy. This is a domain-knowledge-based imputation, not a generic fill.
Columns with no analytical value	Host name, house rules, license, listing name	Removed. These are either personal identifiers, unstructured free text, or largely empty fields that add no value to structured analysis.

5. Four Smart Techniques Used

Beyond straightforward fixes, this project used four technically sophisticated approaches that are worth understanding at a business level — because they reflect real analytical thinking, not just mechanical cleaning.

Forward-Fill Imputation Using Window Functions

Four columns had scattered missing values: host verification status, borough, instant bookable flag, and construction year. Rather than filling all blanks with the most common value (which can introduce systematic bias), we filled each blank with the value from the row immediately above it in the table. This works well for Airbnb data because listings from the same host or area tend to appear in sequence — adjacent rows are genuinely similar. A seed default value handled the very first row where no previous row existed.

► **Smarter than mode imputation: uses local context rather than a global average.**

Context-Aware Cancellation Policy Imputation

When cancellation policy was missing, we did not simply fill with the most common policy. Instead, we used the logically related `instant_bookable` column as a guide: if a listing is instantly bookable (guests can book without host approval), it almost certainly uses a flexible cancellation policy. If it requires host approval before booking, it almost certainly uses a strict policy. This domain knowledge — how Airbnb's own platform logic works — makes the imputation defensible and accurate.

► **Context-aware logic produces more accurate fills than any statistical method.**

Currency String Parsing

Prices were stored in the database as text strings like '\$1,200' rather than as numbers. This is a common issue with data exported from booking platforms. A two-step process removed the dollar sign (always in position 1) and the comma separators, then converted the column to an integer type. Rows where no price remained after stripping were set to NULL rather than zero — because a missing price is unknown, not free.

► **Numeric columns must be stored as numbers, not formatted text, for any mathematical operation to work.**

Date Parsing and Logical Validation

Review dates were stored as text in the format month/day/year (e.g. '3/15/2023'). These were converted to proper calendar dates in two steps: first replacing the slash separators, then applying a format-aware date parsing function. After conversion, all records with a review date in the future were deleted. A future review date is logically impossible — a listing cannot have been reviewed on a date that has not yet occurred. No correction was attempted; these records were simply removed.

► **Date validation catches impossible values that no other check would find.**

6. The Final Clean Dataset

After all six stages, the output table contains 23 clean columns — each with a correct data type, consistent values, and no blank strings masquerading as data. Here is what was retained and what each column now contains:

Column	Data Type	What it contains
Listing ID (Primary Key)	Integer	Unique identifier for each listing — guaranteed unique
Host ID	Integer	Unique identifier for each host
Host verification status	Text	Verified or Not Verified — standardised from two inconsistent formats
Borough	Text	One of NYC's 5 boroughs — no missing values
Neighbourhood	Text	Specific neighbourhood — rows without this were deleted
Latitude	Decimal	Rounded to 2 decimal places
Longitude	Decimal	Rounded to 2 decimal places
Country	Text	United States (defaulted for all listings)
Country code	Text	US (defaulted for all listings)
Instant bookable	Text	TRUE or FALSE — standardised from multiple formats
Cancellation policy	Text	flexible, moderate, or strict — missing values inferred
Room type	Text	Whitespace cleaned; type standardised
Construction year	Integer	Missing values forward-filled
Price per night (USD)	Integer	Currency symbols removed; properly numeric
Service fee (USD)	Integer	Currency symbols removed; properly numeric
Minimum nights	Integer	No changes needed
Number of reviews	Integer	No changes needed
Last review date	Date	Converted from text; future dates deleted
Reviews per month	Decimal	Empty strings converted to NULL
Review rating	Integer	Empty strings converted to NULL
Host listings count	Integer	Empty strings converted to NULL
Availability (days/year)	Integer	Empty strings converted to NULL
Gender	Integer	0=Female, 1=Male (inferred from height averages, documented)

7. Three Business Recommendations

1 Use the Cleaned Dataset as the Sole Source of Truth for NYC Airbnb Analysis

Any analysis built on the raw CSV will inherit its problems: impossible dates, currency symbols in numeric columns, duplicate listings inflating counts, and inconsistent category labels. The cleaned dataset resolves all of these systematically and with documented decisions behind each choice. Dashboards, pricing models, and neighbourhood comparison tools should be built exclusively on the clean output table — not the raw import. The staging pipeline can be re-run at any time on a refreshed data export, making it fully repeatable.

Strategic Impact: *Eliminates the risk of incorrect analysis caused by known data quality issues.*

2 Treat Missing Neighbourhood as a Deletion Criterion — Not an Imputation Target

When a listing has no neighbourhood recorded, there is no reliable way to infer it. Guessing based on nearby listings or borough averages would introduce systematic error into every geographic analysis that follows — neighbourhood-level pricing, occupancy patterns, and host distribution. This project deleted listings with missing neighbourhoods rather than filling them with approximate values. Any future data pipeline processing Airbnb exports should apply the same rule: neighbourhood-null records should be excluded from analysis, not silently filled.

Strategic Impact: *Protects geographic analysis accuracy — an incorrect neighbourhood label is worse than no label.*

3 Build the Next Layer: Analytical Views and Aggregations on Top of the Clean Table

The clean table is the foundation, not the final product. The next step is to build structured analytical views that answer specific business questions — for example: average nightly price by borough and room type; hosts with the most listings and their cancellation policy patterns; seasonal review volume trends by neighbourhood; the relationship between instant bookability and review scores. These views can be built directly in MySQL or exported to Power BI, Tableau, or Python for visualisation. The cleaning work done here makes all of those next steps reliable.

Strategic Impact: *Converts a clean database into actionable business intelligence with minimal additional effort.*

8. Interesting Things We Found Along the Way

Staging tables are better than in-place edits — always.	Every cleaning step was applied in a new table, never by overwriting the original. This means the complete history of every transformation is preserved and reviewable. In a production environment, this approach prevents the most common data engineering catastrophe: irreversible mistakes on the source of truth.
Context-aware imputation is more	Filling missing cancellation policies based on booking type — rather than the most common policy overall — produces values that are logically

defensible than statistical imputation.	consistent with how Airbnb's platform actually works. Statistical imputation (e.g. filling with the mode) would produce a plausible-looking value with no logical basis. When you can use domain knowledge, you should.
An empty string is not NULL — and the difference matters enormously.	Several columns contained empty strings (") rather than NULL values. In SQL, these are treated differently: NULL means 'unknown' and is excluded from calculations automatically; an empty string is a value that counts as data. Leaving empty strings in numeric columns would cause averages and sums to error or produce wrong results. Converting them to NULL was one of the most impactful fixes in the entire pipeline.
Future-dated records are not always obviously wrong at first glance.	Review dates set in the future would not trigger any error in a standard database. They pass type validation, they look like dates, and they appear in queries. Only a logical check — comparing each date to today — reveals them as impossible. This is a class of data quality issue that automated checks often miss: values that are technically valid but contextually impossible.
Dropping columns is a decision, not a default.	Three columns (host name, house rules, license) were removed, and the listing name was dropped at the final step. Each removal was a deliberate decision based on analytical utility — not because cleaning was too difficult. Keeping useless columns adds noise to any downstream analysis and increases storage and processing costs. A clean schema is as important as clean data.

9. Limitations & Next Steps

Known Limitations

- **Row ordering in forward-fill:** The LAG() imputation carried values forward based on the table's natural row order. Without an explicit sort by listing ID or host ID, the order is not guaranteed in MySQL. In a production pipeline, this should be fixed by adding ORDER BY to the window function.
- **Gender inference uncertainty:** The gender encoding (0=Female, 1=Male) was inferred from average height differences between the two groups — a population-level estimate that may misclassify individual records. This is documented but not corrected.
- **Remaining NULLs in price columns:** Listings where price or service fee could not be parsed remain as NULL. Any analysis involving pricing must explicitly handle these missing values.
- **No external validation:** Cleaning correctness was verified through logical consistency checks (no future dates, no impossible BPs, etc.) but not against an independent Airbnb data source.

Recommended Next Steps

- **Build analytical queries:** Average price by borough and room type; most common cancellation policies by neighbourhood; review volume trends over time; hosts with multiple listings and their average ratings.
- **Export to visualisation tools:** The clean table can be connected directly to Power BI, Tableau, or Looker for dashboard creation. The consistent column types and NULL handling make this straightforward.
- **Add ORDER BY to LAG() calls:** For production use, all LAG() window functions should include an explicit ORDER BY id clause to guarantee deterministic row ordering.
- **Schedule pipeline refresh:** The six-stage cleaning pipeline can be re-run on any new Airbnb data export. Scheduling a monthly refresh would keep the clean dataset current with minimal manual effort.

